

Reviewer Pack — Minimal Index of Generalized Symmetries vs Monotone Circuit ...

Entry 751e9f283a7a · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-24 01:10:05 UTC

Minimal Index of Generalized Symmetries vs Monotone Circuit Complexity for 3-CNF

Entry ID: 751e9f283a7a **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-24 01:10:05 UTC

1. Conjecture

Field A (mathematical branch): Symmetry Theory (Generalized Symmetries) **Field B** (complexity object): Complexity Theory: Monotone Circuit Complexity for 3-CNF

Statement:

[“For every instance of a 3-CNF formula, the minimal number of generalized symmetries required to preserve the formula’s satisfiability is $\Theta(1.5^n)$.”, ‘Specifically, if n is the size of the 3-CNF formula, there exists a constant c such that for all instances with more than c generalized symmetries, the monotone circuit complexity grows exponentially.’, ‘The minimal index of generalized symmetries is computable in polynomial time.’]

Rationale (proposer’s reasoning):

[‘Generalized symmetries provide a framework to capture invariants under transformations, which could reveal hidden structures in computational problems.’, ‘The proposed conjecture suggests that the number of symmetries needed to preserve satisfiability might correlate with circuit complexity, potentially leading to novel complexity lower bounds.’, ‘This connection has not been extensively explored and could expose new insights into both symmetry theory and computational complexity.’]

Taxonomy category: MONOTONE_CIRCUIT_COMPLEXITY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 32a5a697af209eac

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The minimal index of generalized symmetries for a 3-CNF formula of size $n \leq 40$ will be measured against its monotone circuit complexity. Support is given if for all seeds, the symmetry index is within $\Theta(1.5^n)$ of the computed value and the monotone circuit complexity does not exceed an exponential growth rate beyond the threshold.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.70	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.70	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.70	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.80	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - symmetry theory AND generalized symmetries AND monotone circuit complexity for 3-CNF - $\theta(1.5^n)$ AND minimal index AND generalized symmetries - computable in polynomial time AND monotone circuit complexity AND 3-CNF

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_3cnf(n):
        clauses = []
        for _ in range(n):
            literals = [random.choice([1, -1]) * (i + 1) for i in range(3)]
            clause = random.sample(literals, 3)
            clauses.append(clause)
        return clauses

    def is_satisfiable(clauses):
        assignment = {i: None for i in range(1, n+1)}
        stack = []
        for clause in clauses:
            found_unassigned = False
            for literal in clause:
                if assignment[abs(literal)] == -literal:
                    return False
                elif assignment[abs(literal)] is None:
                    found_unassigned = True
                    stack.append((literal, assignment))
                    break
            if not found_unassigned:
                continue
        while stack and stack[-1][0] != literal:
            _, assignment = stack.pop()
```

```

        assignment[abs(stack[-1][0])] = None
    if stack:
        assignment[abs(literal)] = -literal
    return True

def compute_symmetry_index(clauses):
    n = len(clauses)
    symmetries = 0
    for i in range(1, n+1):
        for j in range(i+1, n+1):
            if all((i in clause or -i in clause) == (j in clause or -j in clause) for clause in clauses):
                symmetries += 1
    return symmetries

def compute_circuit_complexity(clauses):
    # Placeholder for actual circuit complexity computation
    return len(clauses)

n = random.randint(5, 40)
clauses = generate_3cnf(n)
symmetry_index = compute_symmetry_index(clauses)
circuit_complexity = compute_circuit_complexity(clauses)

c = 1.5 ** n
conjecture_holds = abs(symmetry_index - c) <= 1 and circuit_complexity > c

return {
    "metric_name": "Symmetry Index vs Circuit Complexity",
    "metric_value": symmetry_index,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": "" if conjecture_holds else f"Symmetry index {symmetry_index} does not match {c}"
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_value = math.sqrt(sum((result["metric_value"] - mean_value) ** 2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample={result['counterexample']} first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
ex vs Circuit Complexity', 'metric_value': 156, 'instances_tested': 1, 'conjecture_holds': False, 'co
TRIAL: {'metric_name': 'Symmetry Index vs Circuit Complexity', 'metric_value': 108, 'instances_tested'
TRIAL: {'metric_name': 'Symmetry Index vs Circuit Complexity', 'metric_value': 69, 'instances_tested'
TRIAL: {'metric_name': 'Symmetry Index vs Circuit Complexity', 'metric_value': 9, 'instances_tested':
TRIAL: {'metric_name': 'Symmetry Index vs Circuit Complexity', 'metric_value': 234, 'instances_tested'
TRIAL: {'metric_name': 'Symmetry Index vs Circuit Complexity', 'metric_value': 193, 'instances_tested'
TRIAL: {'metric_name': 'Symmetry Index vs Circuit Complexity', 'metric_value': 18, 'instances_tested'
TRIAL: {'metric_name': 'Symmetry Index vs Circuit Complexity', 'metric_value': 598, 'instances_tested'
TRIAL: {'metric_name': 'Symmetry Index vs Circuit Complexity', 'metric_value': 81, 'instances_tested'
TRIAL: {'metric_name': 'Symmetry Index vs Circuit Complexi
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Safety rail: critic_challenge_falsified | original: The symmetry index does not match the expected value for multiple instances, indicating that the conjecture's claim about the minimal number of genera | next: Investigate the discrepancy between the computed symmetry indices and the expected values. Consider whether there are specific types of 3-CNF formulas that lead to these mismatches and how they might relate to the monotone circuit complexity.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	10242
2	propose	ollama_remote	glm4:latest	0	0	10395
3	preregistration	ollama_remote	glm4:latest	0	0	5870
4	novelty	ollama_remote	glm4:latest	0	0	4744
5	novelty	ollama_remote	glm4:latest	0	0	7034
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13483
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8973
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9511
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9825
10	judge	ollama_remote	glm4:latest	0	0	10258

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 90335 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in [research/audit/751e9f283a7a.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/751e9f283a7a.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/751e9f283a7a.tar.gz` (if generated)